Project Workflow & Analysis Report

Project: Multimodal Deepfake Detection & Media Forensics Platform

Problem: 02 - AI/ML & Deep Learning Hackathon

Repository: https://github.com/harshsingh0120/-Multimodal-Deepfake-Detection-and-Media-Forensics-Platform

Report date: 2026-11-02

This report has two parts:

- **Part A - Complete build workflow:** everything that was done, in order, and the key decisions.
- **Part B - Anatomy of an analysis:** exactly what the platform's forensic report answers and mentions when it analyses a file.

Important honesty notes (read first)

- **No neural network was trained for this project's core engine.** The forensic engine is hand-written signal-processing math (no learning, no GPU, no dataset).
- **The "trained model" is a *public, pre-trained* Vision Transformer downloaded from Hugging Face** - it was trained by its author, not here. We only load and run it.
- **The only thing "trained" locally is a logistic-regression fusion** over 3 numbers, used for *evaluation only* (not shipped in the product).
- **No API keys are used anywhere.** Every download (Python packages, models, dataset) is public and anonymous. The only credentials involved are the user's own cached Git credentials for pushing.

PART A - Complete Build Workflow

A1. Phase 1 - Understand the problem

- Extracted text from the 3-page problem PDF (`02_deepfake_forensics.pdf`) using `pypdf`.
- Captured all 10 core requirements, advanced challenges, datasets, suggested tech, evaluation metrics, the judging rubric, and the mandatory submission checklist.
- Produced `REQUIREMENTS_ANALYSIS.md` and `REQUIREMENTS_TRACEABILITY.md` (every requirement -> code -> test).

A2. Phase 2 - Environment setup

- Chose **Python 3.12** (better scientific-wheel support than the 3.14 present on the machine); created a `.venv`. **Node.js 24** for the frontend.
- Installed the stack: FastAPI/uvicorn, NumPy/SciPy, OpenCV, scikit-image, scikit-learn, Pillow, soundfile, **librosa**, **imageio-ffmpeg** (bundles ffmpeg - the system had none), ReportLab, Matplotlib.
- **Issue fixed:** OpenCV 5.0 dropped `CascadeClassifier` + bundled cascades -> pinned **OpenCV 4.10.0.84**.

A3. Phase 3 - Backend forensic engine (original code)

Written module-by-module in `backend/app/`:

- `config.py` - all tunables via environment variables.
- `utils/media.py` - file-type detection, frame sampling (OpenCV), audio demux (bundled ffmpeg -> WAV).
- `analyzers/image_analyzer.py` - **ELA**, **FFT** spectral artifacts, **noise-residual** & **sharpness** consistency, Haar face detection, heatmaps.
- `analyzers/video_analyzer.py` - per-frame analysis + **temporal** consistency (optical flow, flicker, face-box jitter), suspicious-frame flagging.
- `analyzers/audio_analyzer.py` - prosodic micro-variation, MFCC dynamism, spectral flatness, band-limiting; log-mel spectrogram + flagged segments.
- `analyzers/sync_analyzer.py` - **lip-sync**: mouth-openness vs. audio-envelope cross-correlation.
- `analyzers/fusion.py` - confidence-weighted **late fusion** + cross-modal disagreement penalty + **temperature-scaling calibration**; the centralised, safety-worded verdict bands.
- `analyzers/model_plugin.py` - optional drop-in trained-model interface.
- `report/pdf_report.py` - the downloadable PDF (ReportLab).
- `main.py` - FastAPI endpoints (`/api/analyze`, `/api/report/{id}`, `/api/health`, `/api/config`).

A4. Phase 4 - Frontend (React + Vite + TypeScript + Tailwind)

- Drag-and-drop upload, staged loader, verdict gauge, per-modality cards with evidence bars, heatmaps, flagged frames/segments, and a PDF-download button. Responsive. Built clean (fixed TS config issues).

A5. Phase 5 - Test & verify

- **28 automated tests** (unit + API + robustness) using deterministic synthetic media.
- Ran the live app in a browser - verified the whole flow (upload -> verdict -> heatmaps -> PDF), **no console errors**. Confirmed the PDF is a valid 705 KB file.

A6. Phase 6 - Documentation & first push

- `README.md`, `TESTING.md`, `FINAL_COMPLIANCE_REPORT.md`, `.gitignore`, `.env.example`, `Dockerfile`, `docker-compose.yml`, run scripts.
- `git init` -> commit -> **pushed to GitHub** (using the user's own cached credentials - no token entered).

A7. Phase 7 - Measuring real accuracy

- Built an **evaluation harness** (`evaluation/metrics.py`, `evaluate.py`) computing **AUC, EER, accuracy, Brier, ECE** for the fused detector vs. single-modality and averaging baselines.
- Downloaded a **public** real/fake face dataset from Hugging Face (anonymous CDN, no key): `djramirezp/face-classification-dataset`.
- **Result (honest):** interpretable engine alone = **AUC 0.273, ~50% (chance)** - it does **not** generalise to this generator. Reported openly (the engine assumes "synthetic extra artifacts", but these GAN faces are *smoother* than real photos, so its signal is weak and sign-flipped).

A8. Phase 8 - Trained-model path ("go higher")

- Installed `torch` (CPU) + `transformers`; wrote a plugin that loads a **public pre-trained ViT** (`dima806/deepfake_vs_real_image_detection`) - **downloaded, not trained here**.
- **Bug found & fixed:** first run gave **AUC 0.22** (worse than chance, *inverted*). Diagnosed a **reversed** `id2label` in the pre-trained model; verified true orientation on known-real faces (sklearn Olivetti); added an `IMAGE_MODEL_FAKE_INDEX` override -> corrected to **AUC 0.780, 75.3%**.
- Added a 2nd public model (`Wvolf/ViT_Deepfake_Detection`), **test-time augmentation**, and **ensemble** -> no AUC gain (models correlated).
- **Learned fusion:** a tiny **logistic regression** over `[ViT-1, ViT-2, engine]`, 5-fold cross-validated -> **AUC 0.820, 78.0%**. The chance-level engine carries *complementary* signal that lifts the trained model by +0.04 AUC - validating the multimodal approach.

A9. Accuracy journey (summary)

Stage	Method	AUC	Accuracy
Baseline	Interpretable engine (hand-coded)	0.273	~50% (chance)
Trained model	Public ViT plugin (label bug fixed)	0.780	75.3%
Ensemble + TTA	Two public ViTs	0.782	75.7%
Best	**Learned fusion (ViTs + engine), 5-fold CV**	**0.820**	**78.0%**

Measured on 300 real/fake faces, cross-dataset, no test-set tuning. Full detail in `EVALUATION.md`.

A10. Tools, sources & credentials used

Thing	Source	Credential
Python packages	PyPI (`pip`)	none
Pre-trained ViT models	Hugging Face Hub	**none** (public, anonymous)
Evaluation dataset	Hugging Face datasets-server CDN	**none** (public)
ffmpeg	bundled via `imageio-ffmpeg`	none
Code hosting / push	GitHub (user's repo)	user's own cached Git credentials

There are zero API keys in this project. `.env.example` contains configuration (ports, limits, model ids) - not secrets - and the core engine needs none.

PART B - Anatomy of an Analysis (what the report answers & mentions)

When you upload a file, the platform returns a structured **JSON assessment** (shown in the UI) and a **downloadable PDF**. Here is every section and the question it answers.

B1. Top-level verdict - "Overall, how suspicious is this?"

Field	Answers	Notes
`fusion.overall_score` (0-1)	*How much evidence of manipulation overall?*	0 = none, 1 = strong
`fusion.verdict_band`	*Which risk category?*	**Low -> Guarded -> Elevated -> High -> Very High**
`fusion.verdict_text`	*Plain-language meaning*	Always worded as **"evidence of potential manipulation", never proof**
`fusion.calibrated_confidence` (0-1)	*How sure is the system?*	Temperature-scaled so it isn't over-confident
`fusion.cross_modal_consistency`	*Do the modalities agree?*	Low value = the modalities disagree (itself suspicious)
`fusion.weights`	*How much did each modality count?*	The late-fusion weighting used
`fusion.rationale`	*Why this overall number?*	Step-by-step explanation of the fusion
`safety_notice`	*The responsible-AI disclaimer*	"evidence of potential manipulation, never definitive proof"

B2. File metadata - "What exactly was analysed?"

`media.filename`, `media_type` (image/video/audio), `size_bytes`, `duration_sec`, `width`/`height`, `fps`, `has_audio`, `sample_rate`. `processing_ms` = how long the analysis took.

B3. Per-modality findings - "Which part is suspicious, and why?"

For **each** of image / video / audio / lip-sync the report gives a block that answers:

Field	Answers
`modality` + `available`	*Which branch, and could it run?* (e.g. lip-sync needs a face + audio)
`score` (0-1)	*Is this modality individually suspicious?* (separate image/video/audio/sync scores)
`confidence`	*How reliable is this modality's score here?*
`summary`	*One-line verdict for this modality*
`evidence[]` -> `name`, `detail`, `contribution`	***Why?*** the specific indicators that fired, each with how much it contributed
`features{}`	*The raw measurements* behind the score (e.g. `fft_highfreq_ratio`, `rms_cv`, `lipsync_best_correlation`)
`heatmaps[]`	***Where?*** visual overlays - ELA map, FFT spectrum, audio spectrogram
`flagged_frames[]`	***When?*** suspicious video frames (timestamp + thumbnail + score)
`flagged_segments[]`	***When?*** suspicious audio time-ranges
`notes[]`	*Caveats for this file* (e.g. "no face detected - facial checks skipped")

What each modality is actually checking

- **Image / visual:** recompression inconsistency (ELA), GAN frequency artifacts (FFT), face-vs-background noise & sharpness mismatch. -> *"Does this still frame show editing/synthesis artifacts?"*
- **Video / temporal:** per-frame artifacts + flicker, optical-flow irregularity, face-box jitter. -> *"Is the face unstable across time the way deepfakes flicker?"*
- **Audio / voice:** flat prosody, low MFCC variation, vocoder flatness, band-limiting, clean noise floor.

-> *"Does this voice sound synthesised/cloned?"*

- **Lip-sync:** correlation between mouth movement and speech loudness. -> *"Do the lips match the audio, or was audio dubbed/cloned over the video?"*

B4. Limitations - "What can this NOT tell you?"

`limitations[]` always lists the honest boundaries, e.g.:

- Scores are forensic **indicators, not proof**.
- Heavy compression/resizing can hide artifacts.
- A "Low" result is **not** a guarantee of authenticity.
- Face-dependent checks need a visible frontal face.

B5. The downloadable PDF - "A shareable evidence record"

`/api/report/{id}` returns a PDF containing: the verdict box (band, score, confidence) with the safety disclaimer, file metadata, a per-modality table, each modality's evidence bullets + embedded heatmaps/spectrogram, flagged frames/segments, the fusion rationale, and the limitations - i.e. the same answers above, formatted for an analyst to attach to a case.

B6. The guiding principle of every report

Every number is an **interpretable indicator with a stated reason**, and every verdict is framed as **"evidence of potential manipulation," never a definitive judgement** - matching the problem statement's safety mandate and the responsible-AI requirement.

One-paragraph summary

A forensic engine was **hand-coded from scratch** (classical image/audio/video math - no training, no keys), wrapped in a FastAPI backend and a React frontend, fully tested, and shipped to GitHub. Accuracy was then **measured honestly** on a public dataset (engine alone = chance), **improved** by plugging in a free public pre-trained ViT (AUC 0.78, after fixing a label bug in that model), and **pushed further** to **AUC 0.82** by learning to fuse the models with the engine's complementary signal. Every analysis the platform produces answers *how suspicious, in which modality, why, where, when, how sure,* and *with what limitations* - always as evidence, never proof.